

AI-typicality reader for pasted text

Build plan for extending readwithjev · draft, 23 September 2026

Goal. A user pastes their own text, scrolls through it, and each sentence near the viewport is scored for *AI-typicality* with Jev: a background tint per sentence, light shading on individual words, and a whole-text strip in place of the book map. **Prerequisite:** this plan starts only after the OpAI-Bench validation has passed its go/no-go criteria and the scoring setup has been frozen.

1. Entry criteria from the benchmark validation

Carry these into the build as fixed inputs. If any fails, fix the method before building UX.

- Sentence-level detection on the OpAI-Bench test split beats a statistical baseline (Fast-DetectGPT or Binoculars) and sits alongside the benchmark's LLM-as-detector baselines.
- Performance holds on the held-out generator (Qwen3-8B) and on the hard mixed-authorship versions (around v4).
- Calibration is within an agreed tolerance on a reliability diagram, and the false-positive rate on human-only text (OpAI-Bench v0) is documented at the chosen default threshold.
- Frozen artifacts: Jev model version, prompt wording, sentence segmenter, prefix window, content-word filter, calibration map and per-genre human baselines.

2. What carries over from readwithjev

Existing piece	Reuse for pasted text
Viewport scoring after a 50 ms scroll pause	Same trigger; only sentences near the viewport are ever scored
"How emotional is JEV" threshold dial, re-ranked in the browser	A strictness dial over calibrated AI-typicality; moving it issues no new requests
Hover names the emotions; colour never the only signal	Hover shows the score band, confidence and the words that contributed most
Whole-book minimap	AI-typicality strip over the whole paste for quick navigation
Request-digest cache, Neon shared store	Digest folds in text, model, prompt and calibration version; stores scores only
Global concurrency lease (JEV_MAX_CONCURRENT_CALLS)	Unchanged
Dry-run and --max-requests benchmark harness	Runs the OpAI-Bench validation and later regression checks
Mocked-JEV browser tests	Extended to the new layer so UI tests cost nothing
Task namespaces; predictions never used as gold	New namespace reader-sentence-ai-typicality-v1, kept apart from benchmark caches

3. What changes because content is pasted

- **Ingestion.** A paste mode replaces the document selector. Normalise whitespace and quotes, segment with the exact segmenter used in validation, and show the sentence count and a request estimate before any scoring.
- **No curated metadata.** The character layer is off. Genre is unknown, so the user picks one, or a single Jev Choice over the genre list proposes it; low confidence falls back to a generic baseline and says so.
- **Normalisation.** Display each sentence relative to the human baseline for its genre, not the raw score, so plain conventional prose does not light up.

- **Privacy.** Pasted text may be unpublished or personal. Never persist raw text server-side; the cache holds digest-to-score rows only. Decide whether the shared cache is used for pasted text at all or kept per session, and state retention plainly in the UI.
- **Cost and abuse.** Cap characters per paste, set a per-session request budget and a per-IP rate limit, and keep the global lease. Lazy scoring means unread text costs nothing.
- **Adversarial text.** Prefixes go into question instructions, so pasted text can try to steer the classifier. Quote and escape it, and keep a test set of injection strings.

4. Scoring request (per visible passage)

One Jev request per passage near the viewport, or several passages batched together. Each content word gets a Noul whose instructions carry its prefix (the last ~30 words, including the end of the previous sentence); the state stays neutral, holding only the genre, so later words never leak in. Add two or three passage-level stylistic Nouls. The server aggregates calibrated word scores into a sentence score, applies the genre baseline, and returns the sentence score, word scores and an “insufficient information” flag. Check the per-request question limit on TypeSafe's Models page and split long passages if needed.

5. Display rules

- Label everything “AI-typicality”, never “AI-written”, and keep the README's stance that no highlight is a measurement.
- Grey out low-information sentences: very short lines, dialogue fragments, quotations and lists.
- No document-level verdict at launch. Smoothing across sentences is off by default and, if offered, labelled as a display choice, since it can hide a single edited paragraph.

6. Phases

Phase	Work	Done when
1. Scoring service	Endpoint, frozen artifacts, cache namespace, calibration and genre normalisation	Replaying the OpAI-Bench test slice through the endpoint reproduces the validation metrics
2. Paste ingestion	Paste mode, segmentation, limits, request estimate, genre step	Browser tests pass against mocked responses
3. Reader layer	Sentence tint, word shading, strip map, strictness dial, hover detail	Mocked demo and accessibility checks pass on desktop and mobile drawers
4. Hardening	Privacy and retention, budgets, injection tests, false-positive audit	False-positive rate on the 219k-sentence Gutenberg corpus is documented at the default threshold
5. Pilot	Small test with pastes of known provenance (human, AI, mixed)	Pilot report, including where readers over-trusted the tint

7. Risks and open questions

- **Model drift.** Pin the Jev version (not `jev-latest`) in the digest; any upgrade means recalibrating.
- **Memorised classics.** Famous public-domain texts may look AI-typical because models know them well. Report their false-positive rate separately in the Gutenberg audit.
- **Genre mismatch.** A pasted genre with no human baseline will be miscalibrated; flag it rather than guess.
- **Mixed authorship.** Partly edited text is the hardest case in OpAI-Bench, and also the most common real use.
- **Misuse.** The likeliest harmful use is accusing a student or writer. The copy, the lack of a verdict and the grey-out rules exist for this reason.